

LED Control

LED Control

- 1. LED Lighting Overview
- 2. Core Concepts
 - 2.1 Common-Anode Connection
 - 2.2 Core API — `machine.Pin`
 - 2.3 `time.sleep()` — Blocking Delay
- 3. Hardware Dependencies
 - 3.1 ESP32-S3 GPIO Hardware Features
- 4. Project Architecture and Flow
- 5. Source Code Details
 - 5.1 Import Modules
 - 5.2 Initialize GPIO
 - 5.3 Main Loop — Blink Logic
- 6. Operation Procedure
 - 6.1 Flashing and Running
 - 6.2 Normal Phenomenon
 - 6.3 Verification Methods
- 7. Common Problems

1. LED Lighting Overview

GPIO (General Purpose Input/Output) is the most basic peripheral interface of an MCU. Through software configuration, the chip pins can be set to **input mode** (reading external signals) or **output mode** (driving external circuits).

This experiment makes two onboard LEDs blink synchronously at 1Hz, demonstrating the basic usage of the MicroPython `machine.Pin` module — only 6 lines of core code are needed to complete GPIO output control.

Typical application scenarios:

Application Type	Example
Digital output	LED blinking, relay control, buzzer driving
Digital input	Button detection, switch status reading, sensor digital signals
Bidirectional I/O	I2C bus (SDA), single-wire bus protocol
Interrupt input	External event triggering
Analog input	ADC sampling
Multiplexed function	UART / SPI / I2C / PWM peripheral signals

2. Core Concepts

2.1 Common-Anode Connection

The onboard LEDs use a **common-anode** connection: the anode is fixed to 3.3V, and the cathode goes to the GPIO through a current-limiting resistor.

```
VDD(3.3V) → LED anode → LED cathode → current-limiting resistor → GPIO pin
```

GPIO Output	LED State	Principle
0 (Low)	On	GPIO internally pulled to GND, current flows through the LED
1 (High)	Off	GPIO is at the same potential as VDD, no current

`led.value(0)` = on, which is the opposite of the intuitive "1 = on". Refer to the table above to understand.

2.2 Core API — `machine.Pin`

MicroPython encapsulates all GPIO operations in the `machine.Pin` class. Initializing a `Pin` object completes the pin configuration.

Constructor:

```
from machine import Pin
led = Pin(pin_id, mode, pull=None)
```

Parameter	Description	Possible Values
<code>pin_id</code>	GPIO number	ESP32-S3: 0~48
<code>mode</code>	Pin mode	<code>Pin.OUT</code> output / <code>Pin.IN</code> input / <code>Pin.OPEN_DRAIN</code> open-drain
<code>pull</code>	Pull-up/down resistor	<code>None</code> (default) / <code>Pin.PULL_UP</code> / <code>Pin.PULL_DOWN</code>

Common methods:

Method	Description	Example
<code>value(n)</code>	Write/read level. With argument = write, without = read	<code>led.value(0)</code> outputs low level
<code>on()</code> / <code>off()</code>	Convenience methods. <code>on()</code> = high level, <code>off()</code> = low level	<code>led.on()</code>
<code>toggle()</code>	Toggle the output level	<code>led.toggle()</code>

2.3 `time.sleep()` — Blocking Delay

```
import time
time.sleep(0.5)      # second-level delay, blocks for 0.5s
time.sleep_ms(500)   # millisecond-level delay, blocks for 500ms
time.sleep_us(100)   # microsecond-level delay, blocks for 100µs
```

Under the hood it is FreeRTOS `vTaskDelay()`: the task enters the Blocked state and yields the CPU, then wakes up automatically when the time expires. **Do not busy-wait with a `for` loop** — it will starve the IDLE task and trigger the watchdog.

3. Hardware Dependencies

GPIO	Peripheral	Mode	Description
43	LED0	<code>Pin.OUT</code>	Onboard LED, common-anode, 0=on
44	LED1	<code>Pin.OUT</code>	Same as above

Both LEDs are onboard; no external components are required.

3.1 ESP32-S3 GPIO Hardware Features

Feature	Parameter
Number of available GPIOs	~45 (depends on package; some are shared with Flash/PSRAM)
Supply voltage	3.3V (not 5V-tolerant)
Maximum output current	~40mA/pin (for long-term stable driving, ≤10mA is recommended)
GPIO matrix	On-chip programmable signal routing network; peripheral signals can be mapped to almost any GPIO
RTC GPIO	GPIO0~21 can hold level and wake up during Deep Sleep
Interrupt capability	Every GPIO can be configured for rising edge / falling edge / both edges / low level / high level interrupts

4. Project Architecture and Flow

Script execution (single-file .py)

```
|
|  └─ from machine import Pin          // import Pin module
|  └─ import time                     // import delay module
|  └─ led1 = Pin(43, Pin.OUT)         // GPIO43 push-pull output
|  └─ led2 = Pin(44, Pin.OUT)         // GPIO44 push-pull output
|
|  └─ while True:                     // infinite loop
|      └─ led1.value(1), led2.value(1) // off (common-anode: HIGH=off)
|      └─ time.sleep(0.5)             // hold 500ms
|      └─ led1.value(0), led2.value(0) // on (common-anode: LOW=on)
|      └─ time.sleep(0.5)             // hold 500ms → back to start
```

5. Source Code Details

Full source code: [5.Source Code](#) → [MicroPython](#) → [1.Basic Peripherals](#) → [1.GPIO_LED](#) → [1.GPIO_LED.py](#)

5.1 Import Modules

```
from machine import Pin # Pin class: GPIO control
import time             # delay functions
```

- `machine` is the most core hardware abstraction layer module in MicroPython; the `Pin` class encapsulates all GPIO operations
- `time` provides blocking delay functions, which call FreeRTOS `vTaskDelay()` under the hood

5.2 Initialize GPIO

```
led1 = Pin(43, Pin.OUT) # GPIO43 push-pull output, onboard LED0
led2 = Pin(44, Pin.OUT) # GPIO44 push-pull output, onboard LED1
```

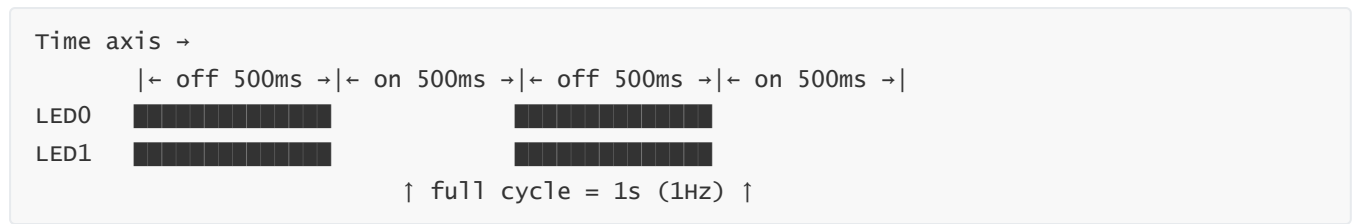
`Pin(pin, Pin.OUT)` calls `gpio_set_direction()` under the hood, setting the GPIO to push-pull output mode with a drive capability of about 10mA — more than enough for an LED.

Any GPIO on the ESP32-S3 can be configured as output through the IO MUX, unlike some MCUs that have dedicated "output pins" restrictions.

5.3 Main Loop — Blink Logic

```
while True:
    led1.value(1) # turn off (common-anode: HIGH=off)
    led2.value(1) # turn off
    time.sleep(0.5) # delay 500ms
    led1.value(0) # turn on (common-anode: LOW=on)
    led2.value(0) # turn on
    time.sleep(0.5) # delay 500ms
```

Execution timing:



The `value()` method directly writes the GPIO output register; a single call takes $< 1\mu\text{s}$. During `time.sleep()`, the task enters the Blocked state, and the system can schedule other tasks or enter IDLE sleep.

6. Operation Procedure

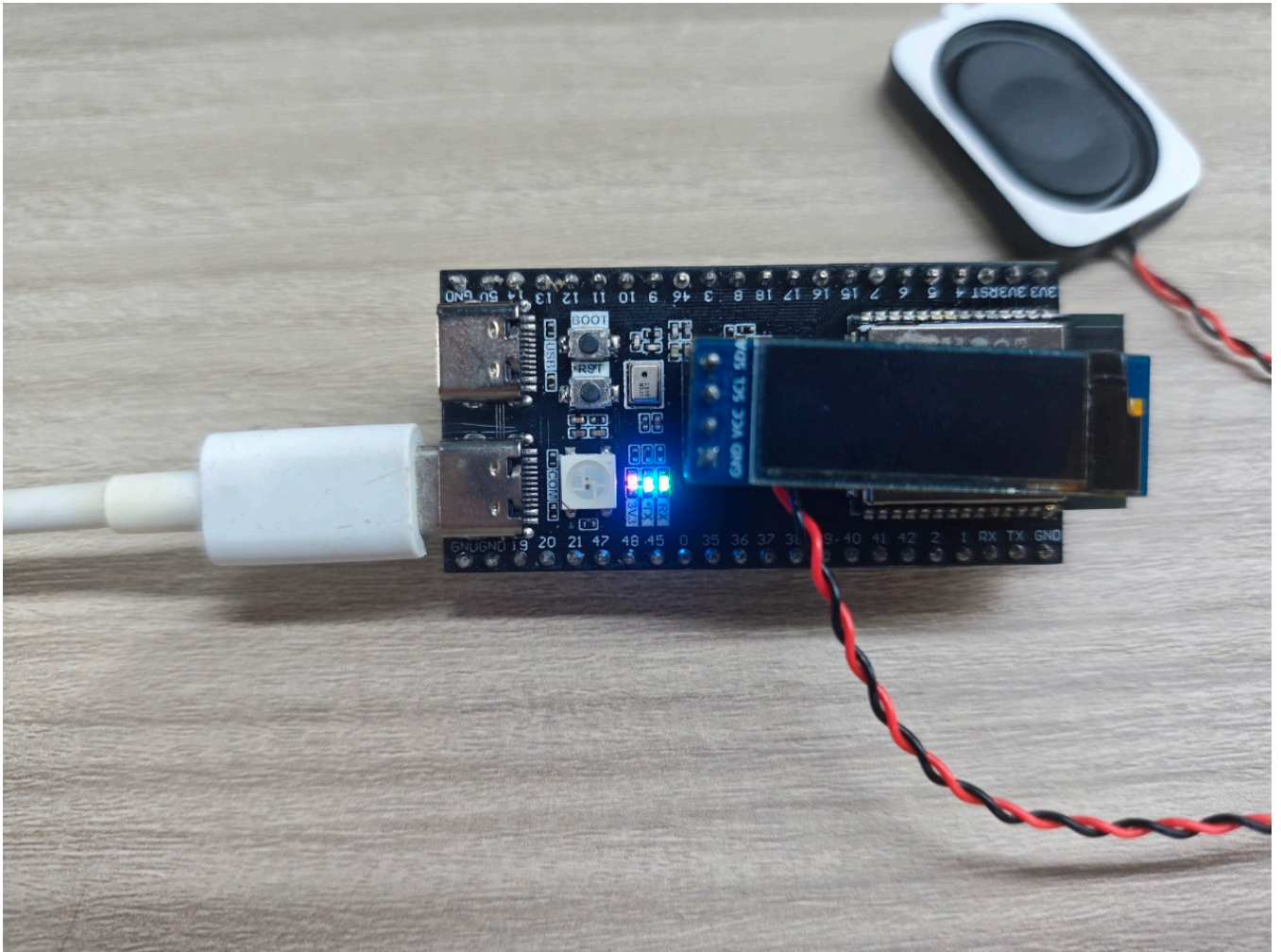
6.1 Flashing and Running

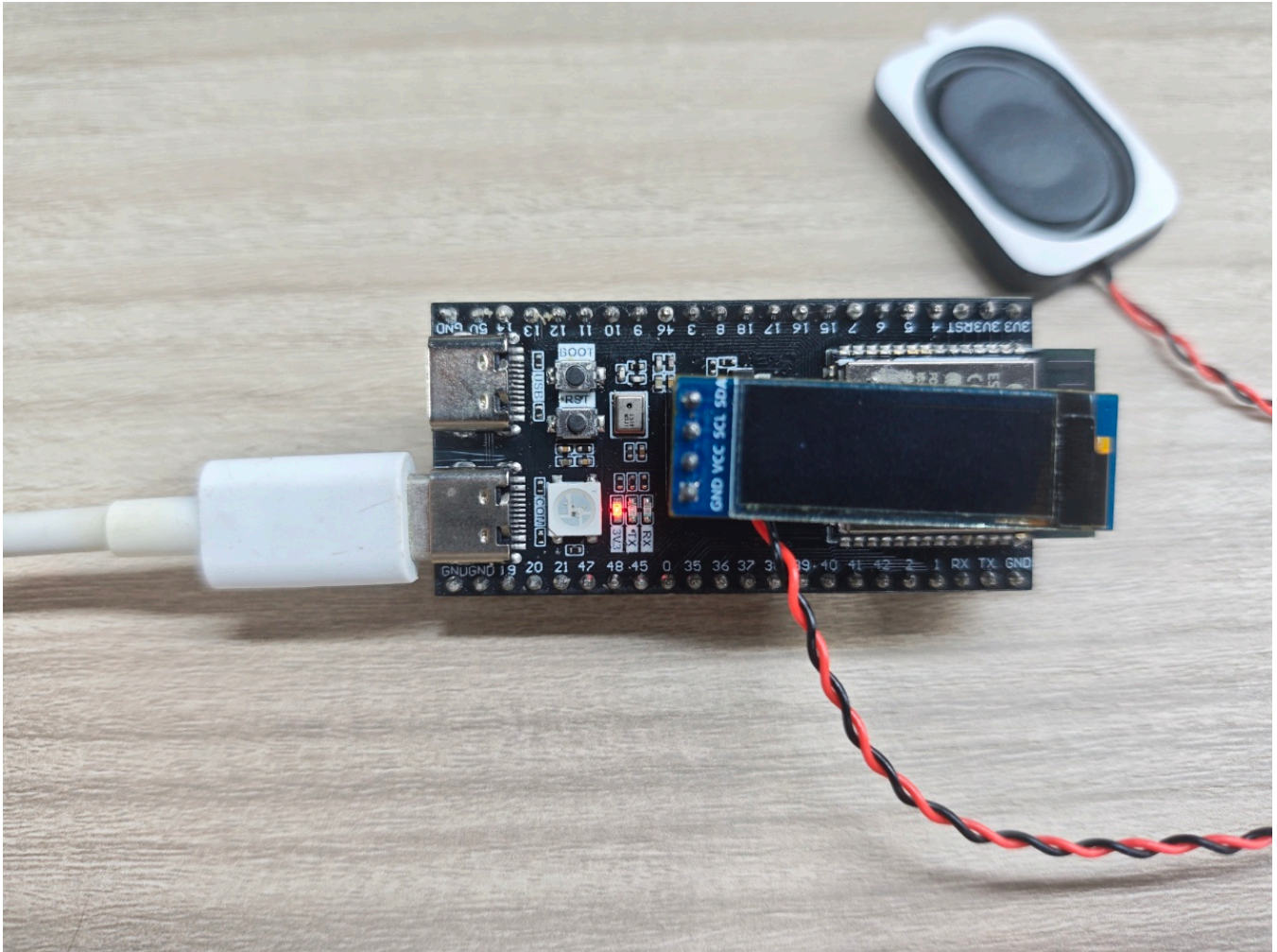
Thonny IDE steps: See the 1. Before Development → 3.Thonny IDE chapter.

1. Connect the development board with a USB cable
2. Select the `ESP32S3` interpreter at the bottom right of Thonny
3. Open `1. GPIO_LED.py` → click the green Run button
4. Observe the onboard LEDs blinking

You can also save the file as `main.py` and upload it to the board; it will run automatically on power-up.

6.2 Normal Phenomenon





After power-up, both LEDs blink **synchronously** at 1Hz: on 500ms → off 500ms → repeat.

6.3 Verification Methods

Operation	Expected Result
Observe after power-up	Both LEDs blink synchronously, cycle ~1s
Change <code>time.sleep(0.5)</code> to <code>time.sleep(0.1)</code>	Blinking speeds up noticeably, about 5Hz
Comment out the two <code>value()</code> lines for <code>led2</code>	Only LED0 blinks
Swap <code>value(0)</code> and <code>value(1)</code>	On/off reversed (off 500ms → on 500ms)

7. Common Problems

Symptom	Cause	Solution
<code>ImportError: no module named 'machine'</code>	PC version of Python is used instead of MicroPython	Confirm the Thonny interpreter is set to ESP32S3
LED does not light up	Pin number does not match the board silkscreen	Confirm GPIO43/44

Symptom	Cause	Solution
LED stays on and never turns off	Common-anode/common-cathode connection is reversed	Swap <code>value(0)</code> / <code>value(1)</code>
Cannot respond to buttons during <code>time.sleep</code>	<code>sleep</code> is a blocking delay	See the Button Detection Experiment for a non-blocking solution
Script runs once and stops	Missing <code>while True:</code> loop	MicroPython has no <code>loop()</code> , you need to write the loop manually
Thonny cannot find the serial port after connecting	Missing USB driver or serial port is occupied	Install CH343/CP210x drivers; close other serial tools
Board keeps restarting after uploading <code>main.py</code>	Syntax error in the script causes startup failure	Run it manually in Thonny first to confirm it works before uploading